

ALU UVM Verification Report

1. Overview

A SystemVerilog UVM-based verification environment was developed to verify the functionality of the parameterized Arithmetic Logic Unit (ALU).

The ALU supports both arithmetic and logical operations and provides status outputs such as carry, overflow, greater-than, less-than, equal and error indications.

The UVM verification environment consists of:

- ALU input agent
- ALU output agent
- Input driver
- Input monitor
- Output monitor
- Sequencer
- Sequence items
- Reference model
- Scoreboard
- Functional coverage
- Assertions

- Environment
- Test
- Virtual interface
- Top-level testbench

The input agent is active and is responsible for generating and driving stimulus to the ALU. The output agent is passive and monitors the ALU output signals without driving any DUT signals.

Both directed and constrained-random sequences were used to verify the ALU functionality under different operating conditions.

The scoreboard compares the actual ALU outputs observed by the output monitor against the expected results generated by the reference model.

Functional coverage was implemented using a dedicated UVM subscriber to ensure that all important ALU operations, modes, input-valid combinations, boundary conditions and relevant cross combinations were exercised.

Concurrent assertions were implemented in the ALU interface to check important design properties such as reset behavior, clock-enable behavior, valid input conditions and status flag relationships.

2. DUT Description

The DUT is a parameterized ALU with a configurable operand width.

The ALU accepts two operands along with operation control signals and generates a result and corresponding status flags.

Main Inputs

- OPA – Operand A
- OPB – Operand B
- CLK – Clock
- RST – Reset
- CE – Clock enable
- MODE – Selects arithmetic or logical operation
- CIN – Carry input
- INPUT_VALID – Indicates validity of input operands
- CMD – Selects the ALU operation

Main Outputs

- RES – ALU result
- COUT – Carry output
- OFLOW – Overflow indication
- G – Greater-than indication
- L – Less-than indication
- E – Equal indication
- ERR – Error indication

The ALU operates in two major modes:

- **Arithmetic mode**
- **Logical mode**

The CMD input selects the specific operation within the selected mode.

3. UVM Verification Architecture

The architecture consists of two agents:

Input Agent

The input agent is an **active agent** and contains:

- Sequencer
- Driver
- Input Monitor

The sequence generates randomized or directed ALU transactions. The sequencer sends the transactions to the driver, which drives the corresponding signals onto the DUT interface.

The input monitor observes the driven transactions and forwards them through an analysis port to the scoreboard and subscriber.

Output Agent

The output agent is configured as a **passive agent**.

It contains:

- Output Monitor

The output monitor does not drive any DUT signals. It only observes the ALU output signals and packages them into transactions.

The output transaction is sent to the scoreboard through an analysis port.

Scoreboard

The scoreboard receives:

- Input transactions from the input monitor
- Output transactions from the output monitor

The reference model calculates the expected ALU result and status flags based on the input transaction.

The expected and actual transactions are then compared to identify functional mismatches.

Subscriber

A subscriber receives transactions through a UVM analysis port and samples the functional coverage model.

Coverage includes:

- Operand A and Operand B coverage values
- Clock enable
- Command coverage
- Mode coverage

- Input-valid coverage
- Carry input coverage
- INP_VALID x Mode
- Inp_valid x cmd
- Cin x cmd
- CE x cmd
- Output flag coverage

Assertions

Concurrent assertions are implemented inside the ALU interface.

The assertions check important design properties such as:

- Reset behavior
- Clock-enable behavior
- Output stability
- Overflow generation
- Carry generation
- Comparison flag behavior
- Valid-input behavior

4. DUT Bugs Identified

Bug 1: Counter increments occur only when INP_VALID=2'b00

Bug 2: Reset incorrectly dependent on CE

Bug 3: G, L and E outputs become z

Bug 4: Multiplication operation is not 3 clock cycle

Bug 5: Incorrect INP_VALID for alu operations with only one variable valid

Bug 6: Incorrect left shift implementation

Bug 7: Incorrect right shift implementation

Bug 8: Incorrect rotate left implementation

Bug 9: Incorrect rotate right implementation implementation

Bug 10: Incorrect increment A implementation

5 Reference Model

A reference model was implemented inside the scoreboard to calculate the expected ALU output independently of the DUT implementation.

The reference model uses:

- Operand A
- Operand B
- Command
- Mode
- Carry input

- Input-valid information

to calculate:

- Expected result
- Carry
- Overflow
- Greater-than flag
- Less-than flag
- Equal flag
- Error indication

The expected transaction is then compared with the transaction captured by the output monitor.

6 Scoreboard Checking

The scoreboard compares the expected and actual transactions.

The following outputs are checked:

- RES
- COUT
- OFLOW
- G
- L
- E

- ERR

A mismatch generates an error message containing the expected and actual values, allowing the DUT behavior to be debugged.

7 Coverage Report

Questa Design Coverage						
Scope: /top/DUV						
Instance Path: /top/DUV						
Design Unit Name: work.ALU_DESIGN						
Language: SystemVerilog						
Source File: alu_top.sv						
Local Instance Coverage Details:						
Total Coverage: 94.44% 94.92%						
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	117	117	0	1	100.00%	100.00%
Branches	73	73	0	1	100.00%	100.00%
FEC Conditions	20	18	2	1	90.00%	90.00%
Toggles	204	183	21	1	89.70%	89.70%

8 Conclusion

A UVM-based verification environment was developed to comprehensively verify the parameterized ALU.

The verification environment included active input and passive output agents, constrained-random stimulus generation, monitors, a reference model, scoreboard, functional coverage, cross coverage and SystemVerilog assertions.

Multiple functional scenarios including arithmetic operations, logical operations, boundary conditions, carry, overflow, comparison flags, input-valid conditions, reset and clock-enable behavior were verified.

The scoreboard provided automated comparison between expected and actual DUT behavior, while functional coverage was used to measure verification completeness.